Implementação de Modelos Analíticos com Python

Guia de revisão · do problema de negócio ao modelo avaliado



cegid Academy

Como usar este guia

O percurso completo, em 6 blocos

01 **O processo — CRISP-DM**
o mapa de qualquer projeto

02 **Aprendizagem e tipos de problema**
regressão, classificação, clustering

03 **Análise (AED) — o que procuramos**
qualidade, distribuições, relações

04 **Preparação dos dados**
sem data leakage

05 **Algoritmos**
qual usar para cada problema

06 **Avaliação e métricas**
medir com honestidade

07 **Treino para o exame**
16 perguntas + soluções

Legenda — o que é importante para o exame

★ SAI NO EXAME

O conceito **já saiu em perguntas** do exame de certificação (banco de questões do Data Analysis Exam).

NO PROGRAMA

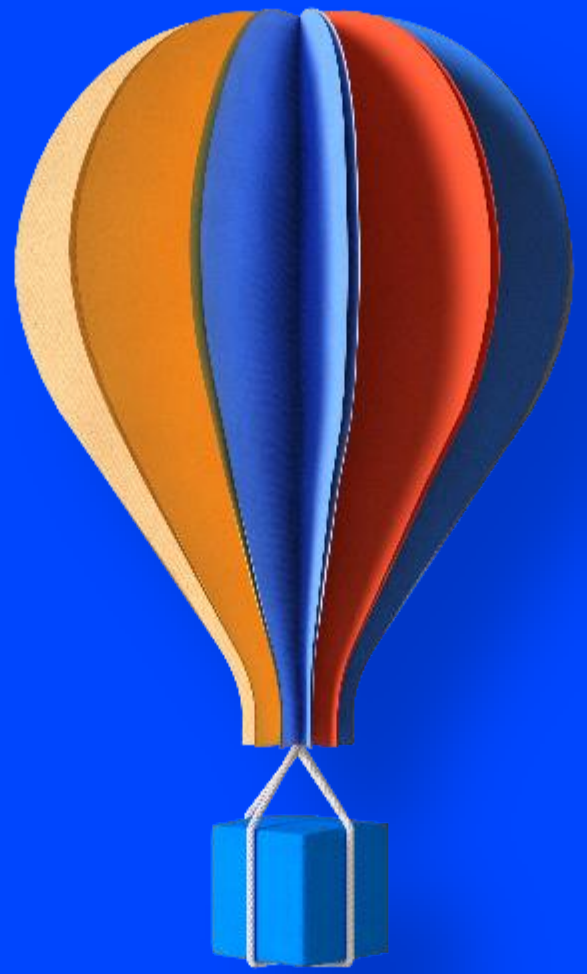
Faz parte do **programa oficial** do exame (Python – Data Analysis) — pode sair.

Texto a laranja = pormenor que costuma ser perguntado (armadilhas das opções de resposta).

`código` = comando pandas / scikit-learn a saber de cor.

01

O processo



CRISP-DM: todas as fases de um projeto analítico, pela ordem certa.

CRISP-DM — o mapa de qualquer projeto

NO PROGRAMA

1 — Negócio

Qual é o problema? O que é sucesso?

Definir alvo e métrica

2 — Dados

Que dados temos? Com que qualidade?

AED · M1–M2

3 — Preparação

Limpar e transformar para o modelo

60–80% do tempo · M3

4 — Modelação

Escolher e treinar algoritmos

M4

5 — Avaliação

Funciona em dados nunca vistos?

M5

6 — Deployment

Pôr em produção e monitorizar

joblib · data drift

É iterativo, não é uma linha reta

Se a avaliação é fraca, volta-se à preparação ou até ao negócio. A AED pode obrigar a rever o problema.

Para fixar

A pergunta central **não** é «que algoritmo uso?» mas «que problema de negócio resolvo?».

Preparação = a fase mais demorada (**60–80%**) — garbage in, garbage out (voltamos várias vezes à fase de análise.

Terminologia e tipos de variáveis

NO PROGRAMA

Conceito	Também se diz	Exemplo (Titanic)
Observação	linha, registo, instância	um passageiro
Variável	coluna, feature, atributo	age, fare, sex
Variável-alvo (y)	target, label	survived
Preditores (X)	features	todas as outras úteis

```
X = df.drop(columns=['survived'])   # preditores
y = df['survived']                  # alvo
```

Primeiro passo de qualquer modelo: **separar X de y**. Nenhuma coluna que «conheça» o resultado pode ficar em X.

Numérica contínua

valores numa escala

`age, fare`

Numérica discreta

contagens inteiras

`sibsp, parch`

Categórica nominal

categorias sem ordem

`sex, embarked`

Categórica ordinal

categorias com ordem

`pclass (1.ª > 2.ª > 3.ª)`

O tipo de variável decide o gráfico (M1), a estatística (M2), o encoding (M3) — e, no alvo, o **tipo de problema**.

02

Aprendizagem e tipos de problema



Antes de escolher um algoritmo, classifica o problema.

Três tipos de aprendizagem automática

NO PROGRAMA

Supervisionada

- Dados: **X e y** (o alvo é conhecido)
- Objetivo: **prever y** para casos novos
- Ex.: sobreviveu? · preço de uma casa

Regressão e classificação

Não supervisionada

- Dados: **só X** (não há alvo)
- Objetivo: **encontrar estrutura** — grupos, padrões
- Ex.: segmentar clientes

Clustering (K-means), PCA

Por reforço

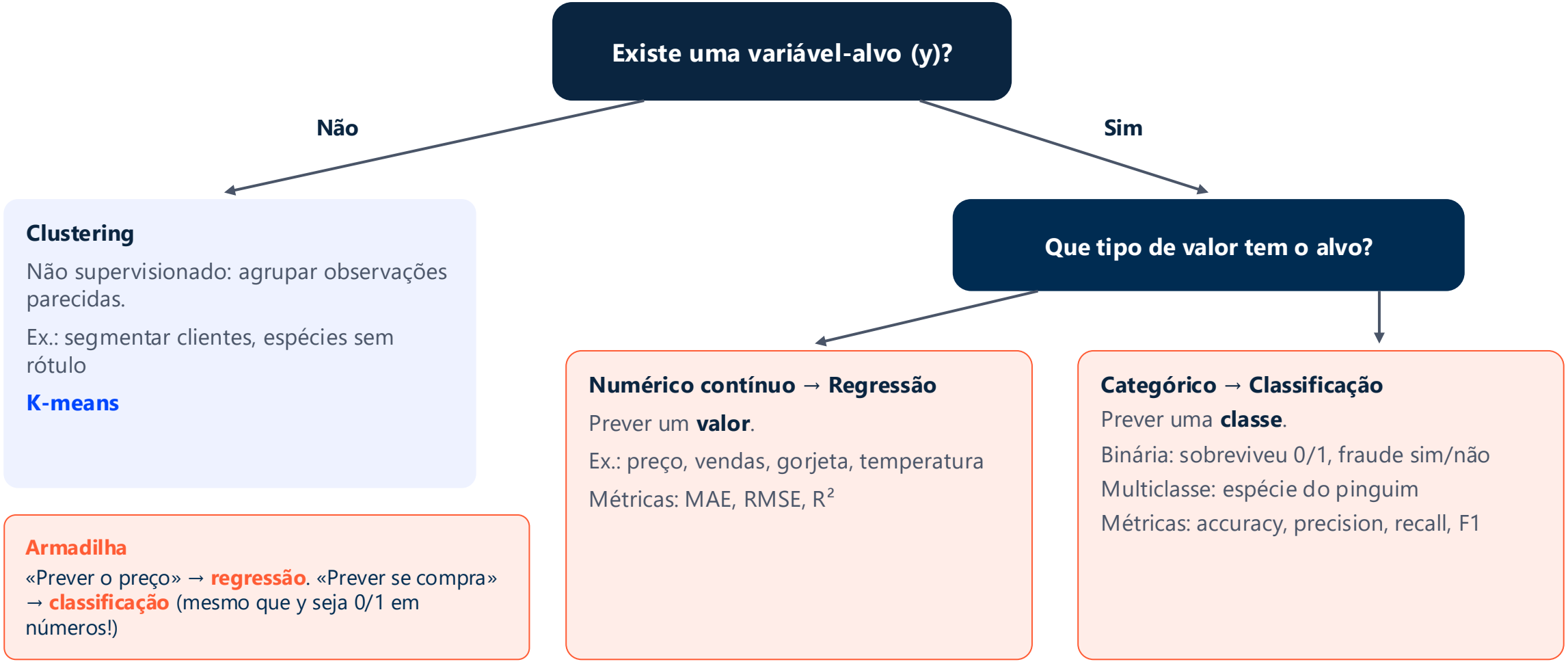
- Dados: **ações e recompensas**
- Objetivo: aprender por **tentativa e erro**
- Ex.: jogos, robótica, carros autónomos

Fora do âmbito deste curso

Pergunta-chave para distinguir: **os dados de treino têm a resposta certa (y)?** Se sim → supervisionada. Se não → não supervisionada.

Que tipo de problema tenho?

NO PROGRAMA



03

Análise: o que procuramos

Cada coisa que descobrimos tem de gerar uma decisão na preparação.



O workflow da AED em 5 passos

★ SAI NO EXAME

1. Estrutura	2. Qualidade	3. Univariada	4. Bivariada	5. Multivariada
Que forma têm os dados?	**Faltam dados? Há erros?**	**Como se distribui cada variável?**	**Como se relaciona com o alvo?**	**Há redundância entre variáveis?**
`df.shape`	**`df.isnull().sum()`**	`df.describe()`	`groupby().mean()`	heatmap da correlação
`df.head()`	`df.duplicated().sum()`	`skew()`	`pd.crosstab()`	`sns.pairplot()`
`df.info()`	**`value_counts()`**	histograma, boxplot	**`df.corr()`**	
`df.dtypes`				

Perguntas que já saíram

`df.info()` → colunas, **tipos e nº de valores não-nulos** (≠ `describe()`, que dá estatísticas: média, desvio, quartis).
`df.isnull().sum()` → **número** de nulos por coluna (não é a percentagem — para % usa `df.isnull().mean()*100`).
`value_counts()` → quantas vezes aparece cada valor · `nunique()` → quantos valores diferentes · `head()` → 5 primeiras linhas.

O que procuramos na AED — e que decisão isso gera

Achado	Como detetar	Decisão na preparação
Valores em falta	`isnull().sum()`, heatmap de nulos	remover ou imputar (mediana, moda, por grupo)
Missing disfarçado	`describe()`: mínimos impossíveis (fare = 0, -1, 999)	converter em NaN e tratar como falta
Duplicados	`duplicated().sum()`	`drop_duplicates()`
Assimetria forte	`skew()` > 1, histograma com cauda	transformar com `np.log1p`; usar mediana
Outliers	boxplot, regra IQR, Z-score	investigar → corrigir, limitar (capping), log ou manter
Muitas categorias / raras	`nunique()`, `value_counts()`	agrupar raras; escolher o encoding
Alvo desequilibrado	`value_counts(normalize=True)`	`stratify=y` no split; métricas além da accuracy
Variáveis muito correlacionadas	`df.corr()`, heatmap (|r| > 0,8)	remover uma, PCA ou regularização
Data leakage	coluna que «já sabe» o resultado (ex.: alive)	remover antes de modelar

Estatística descritiva

★ SAI NO EXAME

Centro

- **Média** — sensível a outliers
- **Mediana** — robusta (valor do meio)
- **Moda** — para categóricas
- Média ≫ mediana → cauda à direita

Dispersão

- **Desvio padrão** — distância típica à média
- **IQR = Q3 − Q1** — metade central dos dados
- **Quartis:** `quantile(0.25)`, `quantile(0.75)`

Forma

- **Assimetria (skew)** — > 0 cauda à direita (ex.: fare)
- **Curtose** — caudas pesadas / muitos extremos
- |skew| < 0,5 ≈ simétrica

```
df.describe()                  # média, desvio, quartis
df['fare'].median()
df['fare'].skew()              # assimetria
df['fare'].quantile([0.25, 0.75])
```

No exame

`quantile()` recebe **proporções entre 0 e 1** (0.25), não 25.

Para variáveis assimétricas, a mediana descreve melhor o «típico» e é a melhor escolha para imputar.

Relações e inferência

★ SAI NO EXAME

- **Correlação** — força e direção da relação, de −1 a 1
- **Pearson** (por omissão): relação **linear**
- **Spearman**: relação monótona, robusta a outliers
- Correlação ≠ causalidade
- **Covariância**: direção da relação, mas sem escala fixa

- **Amostragem** — trabalhar com uma parte representativa
- **Intervalo de confiança 95%** — faixa onde se espera a média da população; amostra maior → IC mais estreito
- **Testes de hipóteses** — t-test / ANOVA (numérica × grupos), qui-quadrado (categórica × categórica); p < 0,05 → diferença significativa

```
df.corr(numeric_only=True)       # matriz
df['a'].corr(df['b'])            # entre 2 colunas
df['a'].corr(df['b'], method='spearman')
df.cov(numeric_only=True)        # covariância
```

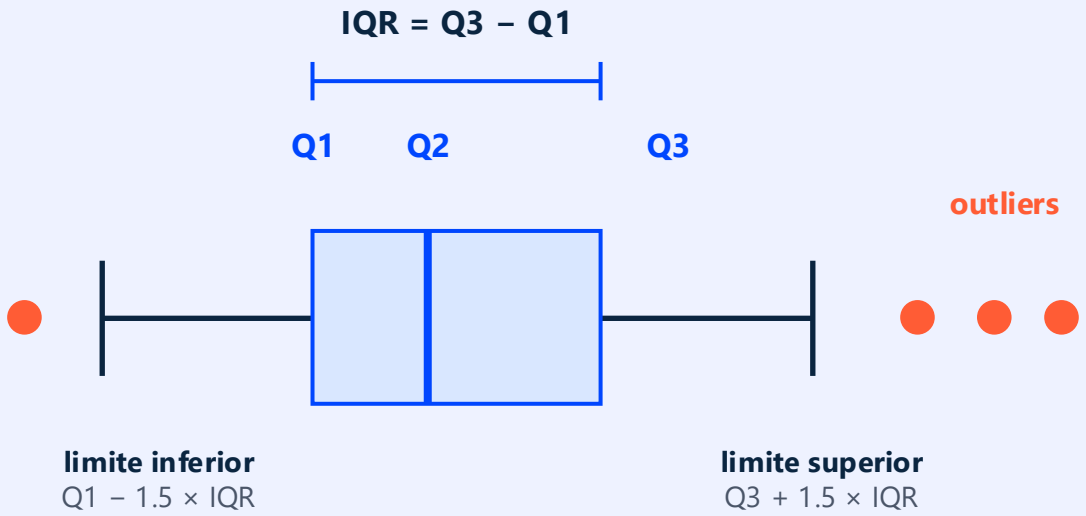
No exame

Correlação entre 2 colunas: `df['a'].corr(df['b'])` (não `df.corr('a','b')`).

Matriz de covariância: `df.cov()` · amostra de 20%: `df.sample(frac=0.2)` (não `sample(0.2)`).

Outliers — a regra do IQR

★ SAI NO EXAME



Q2 = mediana · o boxplot desenha exatamente esta regra

```
Q1 = df['valores'].quantile(0.25)
Q3 = df['valores'].quantile(0.75)
IQR = Q3 - Q1
lim_inf = Q1 - 1.5 * IQR
lim_sup = Q3 + 1.5 * IQR
out = df[(df['valores'] < lim_inf) | (df['valores'] >
lim_sup)]
```

Z-score — `z = (x - média) / desvio`; outlier se o módulo de z > 3

Assume normalidade e é afetado pelos próprios outliers → o IQR é mais **robusto**.

O que já saiu (3 perguntas!)

Regra: outlier se < Q1 − **1.5**×IQR ou > Q3 + **1.5**×IQR (não 3×IQR, não «média ± desvio», não «z entre −1 e 1»).

Quartis com `quantile(0.25)` e `quantile(0.75)` — não `median(0.25)` nem `quantile(25)`.

Tratamento: **identificar com um método estatístico e depois decidir** (corrigir, limitar com `clip`, transformar com log ou manter). Nunca apagar às cegas.

pandas para análise — cábula do exame

★ SAI NO EXAME

Ver tipos e não-nulos	`df.info()`
Contar nulos por coluna	`df.isnull().sum()`
Frequência de cada valor	`df['c'].value_counts()`
Filtrar linhas	`df[df['idade'] > 30]`
Por rótulo / por posição	`df.loc[r, 'col']` / `df.iloc[0:4]`
Média por grupo	`df.groupby('cidade')['salario'].mean()`
Várias agregações	`df.groupby('p')['v'].agg(['mean', 'sum'])`
Por grupo, mesmo nº de linhas	`df.groupby('g')['v'].transform('mean')`
Tabela dinâmica	`df.pivot_table('v', index='p', columns='m', aggfunc='sum')`
Amostra de 20%	`df.sample(frac=0.2)`
Os 3 maiores valores	`df['vendas'].nlargest(3)`
Texto → data	`pd.to_datetime(df['data'])`
Média móvel de 7	`df['vendas'].rolling(7).mean()`
Juntar tabelas (join)	`pd.merge(a, b, on='id')`

Todas estas operações apareceram em perguntas do exame. Lê as opções com atenção: as erradas costumam ter nomes inventados (`df.correlation()`, `df.normalize()`, `sample(0.2)`).

Que gráfico para que pergunta?

★ SAI NO EXAME

Pergunta	Gráfico	Código
Como se distribui uma numérica?	histograma / KDE	`sns.histplot(df, x='age', kde=True)`
Há outliers? Grupos diferem?	boxplot	`sns.boxplot(df, x='pclass', y='fare')`
Relação entre 2 numéricas?	scatter	`sns.scatterplot(df, x='a', y='b')`
Quantos por categoria?	barras	`sns.countplot(df, x='sex')`
Evolução no tempo?	linha	`df.plot.line(y='Vendas')`
Correlações entre todas?	heatmap	`sns.heatmap(df.corr(), annot=True)`
Todas as relações 2 a 2?	pairplot	`sns.pairplot(df, hue='species')`

Perguntas que já saíram

Biblioteca de **visualização estatística**: **seaborn** (assenta no Matplotlib).

Importar o Matplotlib: `import matplotlib.pyplot as plt`

Guardar em PNG com boa resolução: `plt.savefig('grafico.png', dpi=300)`

Gráfico de linhas a partir do pandas: `df.plot.line(y='Vendas')`

Na AED o gráfico serve para decidir:

cauda longa → log · outliers → IQR · |r| alto → redundância

04

Preparação dos dados

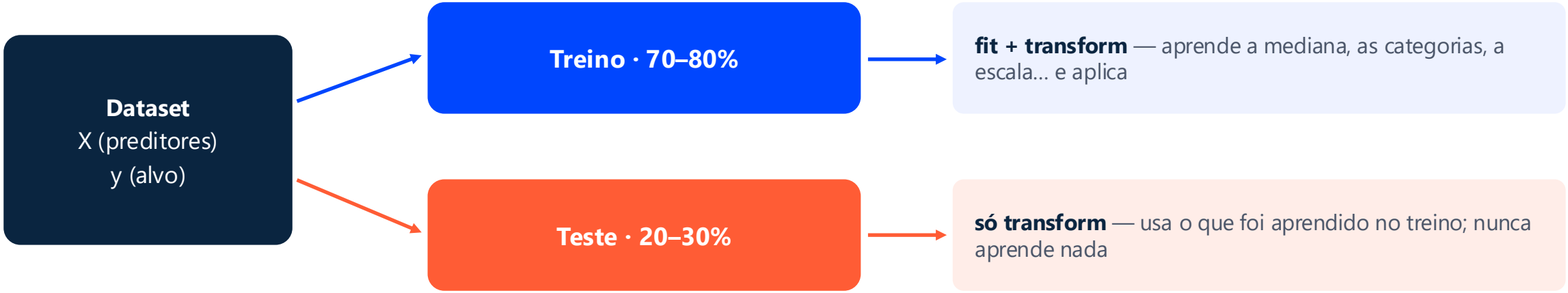
Traduzir os dados para a linguagem do modelo — sem batota.





Regra de ouro: dividir ANTES de preparar

NO PROGRAMA




```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

`test_size=0.2` → 20% para teste · `stratify=y` → mantém a proporção das classes

`random_state=42` → divisão reprodutível (sai sempre igual)

Porquê

Data leakage = informação do teste (ou do futuro) entra no treino → o modelo parece melhor do que é.

Exemplos: calcular a média para imputar com o dataset todo; deixar em X uma coluna que é o alvo disfarçado (`alive`).

O teste serve para **avaliar em dados nunca vistos** — não para treinar nem escolher.

A sequência da preparação

1	**Limpeza**	duplicados, tipos, missing disfarçado, colunas redundantes ou com leakage
2	**Valores em falta**	remover ou imputar (mediana / moda / por grupo)
3	**Outliers**	IQR ou Z-score → corrigir, clip, log ou manter
4	**Encoding**	categorias → números (Label, One-Hot, Frequency, Target)
5	**Feature engineering**	criar variáveis com significado: razões, somas, log, faixas, datas
6	**Scaling**	pôr as numéricas na mesma escala (Standard, MinMax, Robust)
7	**Redução de dimensionalidade**	seleção de variáveis ou PCA

Pipeline

Junta imputação + encoding + scaling numa só «linha de montagem».

fit no treino, transform no teste — anti-leakage automático.

```
prep = ColumnTransformer([
 ('num', num_pipe, num_cols),
 ('cat', cat_pipe, cat_cols)])

prep.fit_transform(X_train)
prep.transform(X_test)
```

Valores em falta

★ SAI NO EXAME

Estratégia	Quando	Cuidado
Remover linhas	poucas linhas com NaN	perde dados
Remover coluna	coluna quase toda vazia (deck: 77%)	perde a variável
Imputar mediana	numérica assimétrica	robusta a outliers
Imputar média	numérica simétrica	sensível a outliers
Imputar moda / constante	categórica; «Desconhecido»	—
Imputar por grupo	grupos informativos (idade por título)	melhor estimativa

```
df.isnull().sum()          # detetar
df.dropna()                # remove linhas
df.fillna(0)               # NaN -> 0
med = X_train['age'].median()
X_train['age'] = X_train['age'].fillna(med)
X_test['age']  = X_test['age'].fillna(med)
```

Já saiu (várias vezes)

`dropna()` **remove** linhas com valores em falta.

`fillna(valor)` **substitui** os NaN por um valor; todos por zero → `df.fillna(0)`.

A mediana é calculada **só no treino** e aplicada aos dois conjuntos.

Encoding — categorias em números

NO PROGRAMA

Estratégia	Quando usar	Cuidado
Label / Ordinal	binárias ou ordinais (ordem real: baixo < médio < alto)	em nominais inventa uma ordem
One-Hot	nominais com poucas categorias (sex, embarked)	cria muitas colunas
Frequency	nominais com muitas categorias	categorias com a mesma frequência colidem
Target / Mean	muitas categorias ligadas ao alvo	leakage: calcular só no treino

Os modelos só trabalham com números: **texto e categorias têm de ser codificados**.

Regra rápida: tem ordem? → Ordinal. Não tem e são poucas? → One-Hot.

One-Hot em ação

embarked
S
C
Q
S

→

emb_S	emb_C	emb_Q
1	0	0
0	1	0
0	0	1
1	0	0

```
pd.get_dummies(df, columns=['embarked'])
OneHotEncoder(handle_unknown='ignore')
```

Scaling — a mesma escala para todas

★ SAI NO EXAME

StandardScaler

```
z = (x − média) / desvio
```

→ média 0 e desvio 1

uso geral; sensível a outliers

MinMaxScaler

```
(x − min) / (max − min)
```

→ intervalo [0, 1]

muito sensível a outliers

RobustScaler

```
(x − mediana) / IQR
```

→ centrado na mediana

robusto a outliers

Precisa de scaling (usa distâncias ou coeficientes):

KNN · K-means · regressão logística · SVM · redes neuronais · PCA

Não precisa: árvores de decisão e Random Forest (dividem por limiares).

Já saiu

«Como normalizar a coluna x para [0,1]?»

→ `(df['x'] - df['x'].min()) / (df['x'].max() - df['x'].min())`

Não é `df['x'] / df['x'].max()` nem `df.normalize('x')` (não existe).

Criar e reduzir variáveis

★ SAI NO EXAME

Criar variáveis com significado

```
df['family_size'] = df['sibsp'] + df['parch'] + 1
df['is_alone'] = (df['family_size'] == 1).astype(int)
df['fare_pp'] = df['fare'] / df['family_size']
df['log_fare'] = np.log1p(df['fare'])
df['faixa'] = pd.cut(df['age'], [0, 12, 18, 60, 100])
df['ano'] = pd.to_datetime(df['data']).dt.year
```

Agregação · razão · transformação (log) · binning · interação · datas.

Boas variáveis valem mais do que um algoritmo mais complexo.

Dica

`np.log1p` reduz a assimetria de variáveis com cauda à direita (ex.: fare).

Menos variáveis, melhor modelo

Seleção

mantém variáveis originais (correlação, variância, SelectKBest)

→ interpretável

Extração: PCA

cria componentes novas que **preservam a maior variância**

→ perde significado

Porquê reduzir: menos overfitting, menos ruído, treino mais rápido.

Já saiu

Reduz a dimensionalidade preservando a maior variância → **PCA**.

Multicolinearidade (preditores muito correlacionados entre si) → **regularização ou redução de dimensionalidade** (ou remover uma das variáveis).

05

Algoritmos

Que modelo usar para cada tipo de problema.



Mapa de algoritmos do curso

NO PROGRAMA

Algoritmo	Problema	Como funciona (numa frase)	Scaling?	Pontos fortes · cuidados
Regressão Linear	Regressão	ajusta uma reta: $y = \beta_0 + \beta_1 x_1 + \ldots$	sim	simples e interpretável · assume relação linear
Regressão Logística	Classificação	calcula a probabilidade de cada classe (curva em S)	sim	apesar do nome, **classifica** · linear
Árvore de Decisão	Ambos	faz perguntas sucessivas (se… então…)	não	interpretável · tende a **overfitting**
Random Forest	Ambos	muitas árvores em amostras diferentes + votação / média	não	robusto e forte · menos interpretável
KNN	Classificação	vê os K vizinhos mais próximos e vota	sim	simples · sensível à escala e ao K
K-means	Clustering	forma K grupos à volta de centros	sim	escolher K: cotovelo e silhueta

Na dúvida: começa por um modelo **simples e interpretável** (linear/logística) como referência e compara com um **Random Forest**.

scikit-learn: o mesmo padrão para todos

NO PROGRAMA

1 Importar

```
from sklearn.ensemble import RandomForestClassifier
```

2 Criar

```
modelo = RandomForestClassifier(random_state=42)
```

3 Treinar

```
modelo.fit(X_train, y_train)
```

4 Prever

```
y_pred = modelo.predict(X_test)
```

5 Avaliar

```
accuracy_score(y_test, y_pred)
```

Regressão	Classificação
LinearRegression	LogisticRegression
DecisionTreeRegressor	DecisionTreeClassifier
RandomForestRegressor	RandomForestClassifier
—	KNeighborsClassifier

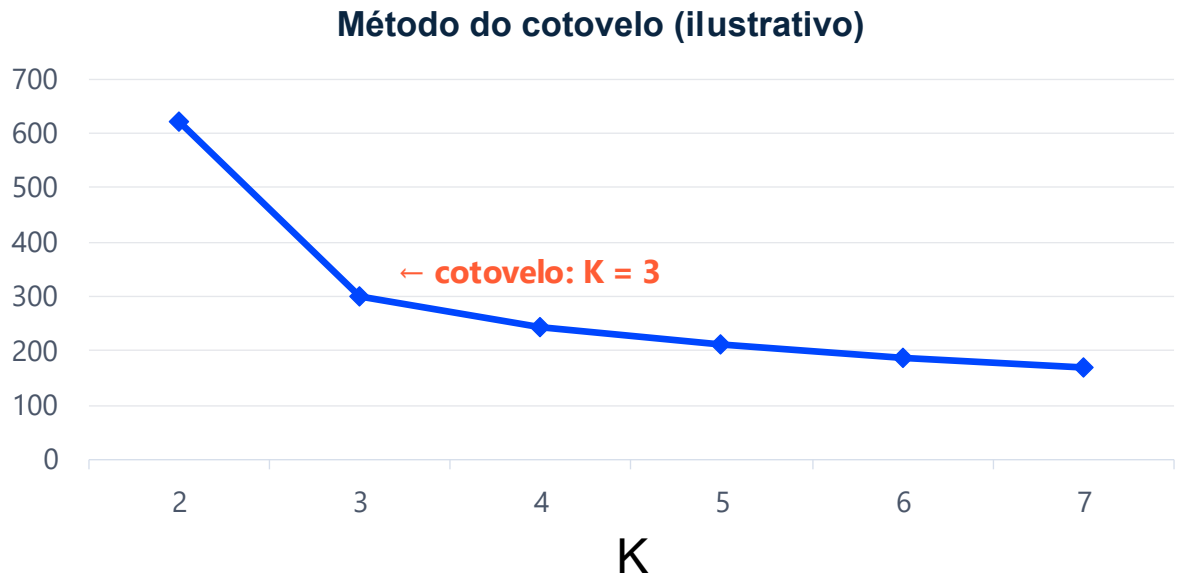
Clustering: `KMeans(n_clusters=3)` → `fit_predict(X)` (sem y!)

Para fixar

`fit` = aprender (só com o **treino**) · `predict` = prever (no **teste** ou em dados novos). Trocar o algoritmo é trocar só a linha «Criar».

Clustering com K-means — e como escolher K

1. Escolher K (nº de grupos)
2. Colocar K centros iniciais
3. Atribuir cada ponto ao centro mais próximo
4. Recalcular cada centro (média do grupo)
5. Repetir 3–4 até estabilizar



Antes: normalizar! O K-means usa distâncias — sem scaling, a variável com números maiores domina.

```
StandardScaler().fit_transform(X)
```

Cotovelo

`inertia_` desce sempre; escolhe o K onde deixa de descer muito.

Silhueta

de −1 a 1: quanto **mais alta**, mais separados os grupos.

06

Avaliação e métricas

Um modelo treinado ainda não é um bom modelo — falta medi-lo.



Underfitting, overfitting e generalização

NO PROGRAMA

Underfitting



Fraco no treino **e** no teste.

Modelo simples demais.

→ **mais variáveis, modelo mais complexo**

Generalização

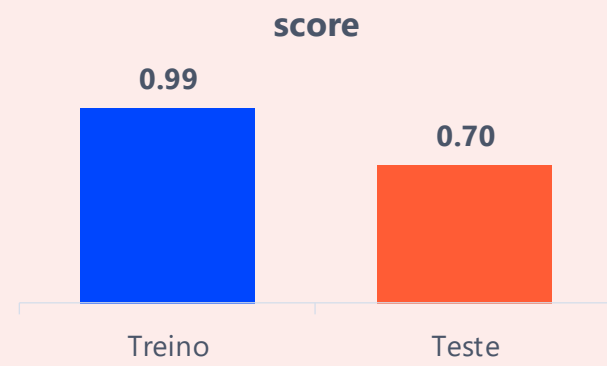


Bom e **parecido** em treino e teste.

É o objetivo.

→ **é este que queremos**

Overfitting



Ótimo no treino, **mau no teste**.

Decorou os dados.

→ **simplificar (max_depth), mais dados, CV**

```
modelo.score(X_train, y_train)  vs  modelo.score(X_test, y_test)     # diferença grande = overfitting
```

Métricas de regressão

NO PROGRAMA

MAE

Erro Absoluto Médio

média de |real − previsto|

Na unidade do alvo: «em média erro 3 €».

↓ **menor é melhor**

RMSE

Raiz do Erro Quadrático Médio

√ média de (real − previsto)²

Penaliza mais os erros grandes. RMSE ≥ MAE.

↓ **menor é melhor**

R^2

Coeficiente de determinação

% da variabilidade explicada

1 = perfeito · 0 = igual a prever sempre a média.

↑ **maior é melhor**

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
mae  = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2   = r2_score(y_test, y_pred)
```

Matriz de confusão e métricas

NO PROGRAMA

		Previsto: Negativo (0)	Previsto: Positivo (1)
Real	Negativo (0)	**VN** verdadeiro negativo	**FP** falso positivo (falso alarme)
Real	Positivo (1)	**FN** falso negativo (falhou!)	**VP** verdadeiro positivo

```
confusion_matrix(y_test, y_pred)
print(classification_report(y_test, y_pred))
```

Accuracy	`(VP + VN) / total`	% de acertos no total
Precision	`VP / (VP + FP)`	dos que previ positivos, quantos eram?
Recall	`VP / (VP + FN)`	dos positivos reais, quantos apanhei?
F1-score	`2·P·R / (P + R)`	equilíbrio entre precision e recall

Armadilha clássica

Com o alvo **desequilibrado** a **accuracy engana**: se 95% são «não», um modelo que diz sempre «não» tem 95% de accuracy e não serve para nada.

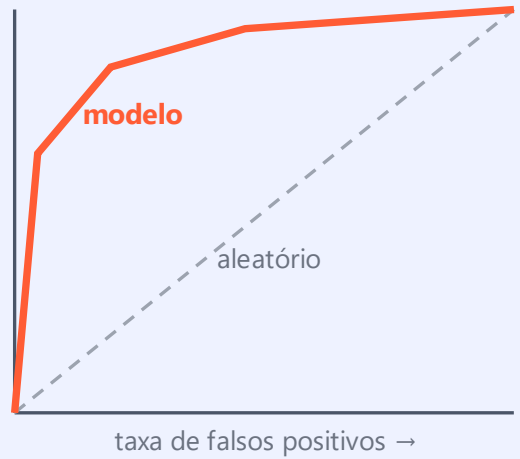
Nesse caso olha para recall, precision e F1.

Que métrica escolher?

Situação	Métrica	Porquê
Falhar um positivo é grave (doença, fraude)	Recall	minimiza FN
Um falso alarme é caro (spam, campanha)	Precision	minimiza FP
Classes desequilibradas, quero equilíbrio	F1-score	junta P e R
Classes equilibradas	Accuracy	simples de explicar
Comparar modelos pela probabilidade	ROC-AUC	independe do limiar
Alvo numérico (regressão)	MAE · RMSE · R^2	ver slide anterior

Escolhe a métrica na fase de **Entendimento do Negócio** — é ela que define o que é «sucesso».

Curva ROC e AUC

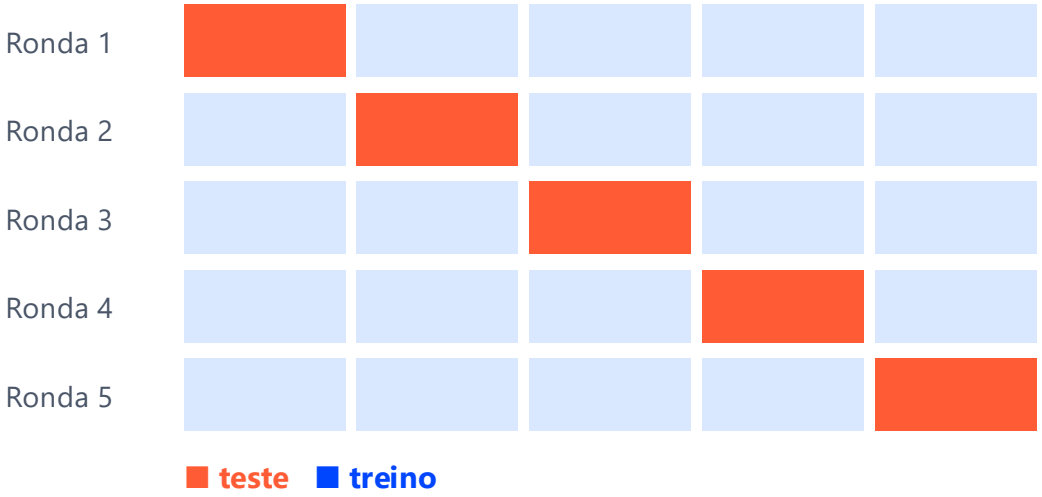


Quanto mais perto do **canto superior esquerdo**, melhor.

AUC: 0,5 = aleatório · 1 = perfeito.

Avaliar com honestidade: validação cruzada e tuning

K-Fold (K = 5) — 5 treinos, 5 notas, usa-se a média




```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(modelo, X, y, cv=cv)
scores.mean(), scores.std()
```

Uma só divisão depende da sorte; a média da CV é mais estável para **comparar modelos**.

Tuning de hiperparâmetros

GridSearchCV — testa todas as combinações (exaustivo, lento)

RandomizedSearchCV — testa combinações ao acaso (rápido, resultado próximo)

Sempre com CV **sobre o treino**.

Deployment

`joblib.dump(pipeline, 'modelo.joblib')` → API/dashboard → monitorizar o **data drift**.

Regra

O conjunto de teste usa-se **uma única vez, no fim**. Se o usas para escolher o modelo, deixa de ser «nunca visto».

07

Treino para o exame



12 perguntas retiradas do exame + 4 perguntas-tipo de modelação.

Treino 1 — explorar os dados

★ SAI NO EXAME

1 **Que método pandas mostra informações gerais do DataFrame, incluindo tipos de dados e valores não nulos?**

- **A** `summary()`
- **B** `details()`
- **C** `info()`
- **D** `describe()`

2 **Qual o resultado de `df.isnull().sum()`?**

- **A** A soma de valores nulos por coluna
- **B** O número de colunas
- **C** A percentagem de valores em falta
- **D** O número total de linhas

3 **Que método calcula a contagem de cada valor numa coluna?**

- **A** `nunique()`
- **B** `count()`
- **C** `value_counts()`
- **D** `unique()`

4 **Qual a diferença entre `.loc` e `.iloc`?**

- **A** `.loc` usa inteiros e `.iloc` rótulos
- **B** Ambos usam só inteiros
- **C** `.loc` usa rótulos e `.iloc` inteiros
- **D** Ambos usam só rótulos

Perguntas retiradas do banco de questões do exame de certificação.

Treino 2 — estatística e outliers

★ SAI NO EXAME

5 **Que comando calcula a correlação entre duas colunas específicas?**

- **A** `np.corr(df['a'], df['b'])`
- **B** `df['a'].corr(df['b'])`
- **C** `df.corr('a','b')`
- **D** `df.correlation(a,b)`

6 **Pela regra do IQR, como se calculam Q1, Q3 e o IQR da coluna `valores`?**

- **A** `median(0.25)` / `median(0.75)`
- **B** `quantile(0.25)` / `quantile(0.75)`
- **C** `mean(0.25)` / `mean(0.75)`
- **D** `quantile(25)` / `quantile(75)`

7 **Qual regra baseada no IQR identifica outliers?**

- **A** Z-score entre −1 e 1
- **B** < Q1 − 1.5·IQR ou > Q3 + 1.5·IQR
- **C** Fora de média ± desvio padrão
- **D** < Q1 − 3·IQR ou > Q3 + 3·IQR

8 **Qual a abordagem mais adequada para tratar outliers antes de aplicar um modelo?**

- **A** Identificar (método estatístico) e decidir
- **B** Remover todas as linhas extremas sem análise
- **C** Ignorar, pois não afetam modelos
- **D** Substituir todos por zero

Perguntas retiradas do banco de questões do exame de certificação.

Treino 3 — preparação

★ SAI NO EXAME

9 **Que método substitui valores ausentes por um valor específico?**

- **A** `fillna()`
- **B** `replace()`
- **C** `dropna()`
- **D** `update()`

10 **Como normalizar a coluna x para o intervalo [0, 1]?**

- **A** `np.scale(df['x'])`
- **B** `df['x'] / df['x'].max()`
- **C** `(x - x.min()) / (x.max() - x.min())`
- **D** `df.normalize('x')`

11 **Que abordagem reduz a dimensionalidade preservando a maior variância possível?**

- **A** One-hot encoding
- **B** Padronização
- **C** PCA
- **D** Normalização

12 **Qual a técnica mais apropriada para lidar com multicolinearidade?**

- **A** Substituir valores ausentes por zero
- **B** Regularização ou redução de dimensionalidade
- **C** Aumentar o número de variáveis
- **D** Remover observações extremas

Perguntas retiradas do banco de questões do exame de certificação.

Treino 4 — modelação e avaliação

NO PROGRAMA

13 **Prever o preço de venda de uma casa é um problema de:**

- **A** Classificação
- **B** Regressão
- **C** Clustering
- **D** Aprendizagem por reforço

14 **Para que serve o conjunto de teste?**

- **A** Treinar o modelo
- **B** Calcular a mediana para imputar
- **C** Avaliar o modelo em dados nunca vistos
- **D** Escolher as variáveis

15 **Um modelo tem accuracy 0,99 no treino e 0,70 no teste. Isto indica:**

- **A** Underfitting
- **B** Overfitting
- **C** Boa generalização
- **D** Um erro de encoding

16 **Num dataset de fraude (2% de fraudes), que métrica é mais informativa do que a accuracy?**

- **A** Accuracy
- **B** R^2
- **C** Recall / F1-score
- **D** MAE

Perguntas-tipo escritas a partir do programa do exame (regressão vs classificação, treino/teste, avaliação básica) — não retiradas do exame.

Soluções

1 **C** `info()` mostra colunas, tipos e não-nulos; `describe()` dá estatísticas.

2 **A** `isnull()` dá True/False; `sum()` conta os True de cada coluna.

3 **C** `value_counts()` diz quantas vezes aparece cada valor.

4 **C** loc = label (rótulo); iloc = integer location (posição).

5 **B** `corr()` aplicado a uma Series recebe a outra Series.

6 **B** Quartis com `quantile()` e proporções entre 0 e 1; IQR = Q3 − Q1.

7 **B** A regra clássica usa 1.5 × IQR.

8 **A** Analisar primeiro; depois corrigir, limitar, transformar ou manter.

9 **A** `fillna()` preenche NaN; `dropna()` remove.

10 **C** Min-Max: o mínimo passa a 0 e o máximo a 1.

11 **C** O PCA cria componentes que preservam a maior variância.

12 **B** Regularização, PCA ou retirar uma das variáveis correlacionadas.

13 **B** O alvo é numérico contínuo.

14 **C** Tudo o que «aprende» usa só o treino.

15 **B** Decorou o treino e não generaliza.

16 **C** A accuracy engana com classes desequilibradas; R^2 e MAE são de regressão.

As 10 ideias para levar para o exame

1. **CRISP-DM:** negócio → dados → preparação → modelação → avaliação → deployment (iterativo).

2. **Alvo numérico** → regressão · **categórico** → classificação · **sem alvo** → clustering.

3. `info()` = tipos e não-nulos · `describe()` = estatísticas · `isnull().sum()` = nulos por coluna.

4. **Outlier** se < Q1 − 1.5·IQR ou > Q3 + 1.5·IQR, com `quantile(0.25)` e `quantile(0.75)`.

5. `dropna()` **remove** · `fillna()` **substitui** · mediana para variáveis assimétricas.

6. **Dividir treino/teste antes de preparar:** `fit` só no treino — senão há data leakage.

7. **MinMax** = (x − min)/(max − min) → [0, 1] · **Standard** → média 0, desvio 1.

8. **PCA** reduz dimensões preservando a variância · multicolinearidade → regularização / PCA.

9. Treino ≫ teste = **overfitting** · ambos fracos = **underfitting**.

10. Regressão: **MAE, RMSE, R^2** · Classificação: **accuracy** (engana se desequilibrado), **precision, recall, F1**.

Números a laranja = temas que já saíram em perguntas do exame.



Bom estudo!

cegid Academy